

SAVORIA RESTAURANT APP MVP

Software Requirements Specification (SRS)

Assignment No. 1 — Restaurant App MVP (Frontend Only, React Native)

Semester: Fall 2026

Application: Savoria Restaurant App MVP

Technology: React Native + Expo Router

Scope: Frontend Only

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements for the Savoria Restaurant App MVP. The application is a frontend-only React Native prototype designed for restaurant customers and restaurant managers.

The purpose of Savoria is to allow customers to browse restaurant menu items, search and filter food, add items to a cart, apply promotional codes, reserve tables, place dine-in or takeaway orders, and track order progress.

The application also provides restaurant managers with a dashboard where they can view incoming orders, manage reservations, and manage menu items.

The MVP demonstrates these functions using React Native, Expo Router, React Hooks, Context API, reducers, custom hooks, local JavaScript mock data, and AsyncStorage. No backend service or real database is used.

1.2 Scope

In Scope

The Savoria Restaurant App MVP includes:

1. Customer login using local mock user data.
2. Customer signup with name, email, password, confirmation password, and role.
3. Manager login using local mock credentials.
4. Menu browsing.
5. Menu categories including Starters, Mains, Desserts, and Drinks.
6. Daily Special identification.
7. Menu item availability display.
8. Menu search with debounce functionality.
9. Category filtering.
10. Menu sorting by price and name.
11. Favourite menu items.
12. Add to Cart functionality.
13. Cart quantity increment and decrement.
14. Remove cart items.
15. Special instructions for cart items.
16. Promotional code application.
17. Order summary calculation.
18. Service charge calculation.

19. Sales tax calculation.
20. Dine-in order selection.
21. Takeaway order selection.
22. Table reservation.
23. Table availability checking.
24. Reservation validation.
25. Reservation confirmation.
26. Reservation cancellation.
27. Order creation using local state.
28. Order tracking.
29. Automatic demo order status progression.
30. Customer profile.
31. Light and dark theme switching.
32. Manager Dashboard.
33. Manager order status management.
34. Manager reservation management.
35. Manager menu management.
36. Local persistence using AsyncStorage.
37. Responsive React Native interface.

Out of Scope

The following are NOT included in this frontend-only MVP:

- Real backend server.
- Real database.
- Firebase authentication.
- Real online payment processing.
- Credit/debit card processing.
- Real-time server communication.
- Push notifications.
- Email notifications.
- SMS notifications.
- Delivery driver tracking.
- Real GPS tracking.
- External restaurant APIs.
- Real payment gateways.
- Production-level security infrastructure.

1.3 Definitions and Acronyms

Term	Definition
SRS	Software Requirements Specification, a document describing system requirements.
MVP	Minimum Viable Product containing the core features needed for the prototype.
UML	Unified Modeling Language used to visually model software systems.
Hook	A React function that allows functional components to use state and other React features.

Context	React mechanism used to share data across components without passing props through every level.
Reducer	A pure function that calculates a new state from the current state and an action.
Mock Data	Local sample data used instead of a real backend or database.
FlatList	React Native component used to efficiently render scrollable lists.
AsyncStorage	Local persistent key-value storage used by the application.
Expo Router	File-based navigation system used by the React Native application.
Component	A reusable UI or software building block in React Native.
Custom Hook	A reusable function beginning with <code>use</code> that contains reusable React logic.

2. Overall Description

2.1 User Roles

Role	Key Permissions & Goals
Customer	Browse menu, search food, manage cart, apply promo codes, reserve tables, place orders, view profile, change theme, and track orders.
Restaurant Manager	View incoming orders, update order statuses, manage reservations, and add/edit/toggle menu items.

2.2 User Stories

Customer User Stories

1. As a customer, I want to browse the restaurant menu, so that I can decide what to order.
2. As a customer, I want to see daily specials, so that I can quickly discover recommended food.
3. As a customer, I want to search menu items, so that I can find food without scrolling through the entire menu.
4. As a customer, I want to filter food by category, so that I can browse relevant dishes quickly.
5. As a customer, I want to check table availability, so that I can reserve a table before arriving.
6. As a customer, I want to book a table for a selected time, so that I can avoid waiting at the restaurant.
7. As a customer, I want to add food to a cart, so that I can prepare my order before checkout.
8. As a customer, I want to apply a promo code, so that I can receive a discount when a valid code is available.
9. As a customer, I want to choose dine-in or takeaway, so that I can select the option suitable for my visit.
10. As a customer, I want to track my order status, so that I know whether my food is pending, preparing, ready, or served.

Restaurant Manager User Stories

11. As a restaurant manager, I want to view incoming orders, so that I can manage customer orders.
12. As a restaurant manager, I want to update order statuses, so that customers can see the progress of their orders.

13. As a restaurant manager, I want to manage reservations, so that I can confirm or decline customer bookings.
 14. As a restaurant manager, I want to add menu items, so that I can update the restaurant menu.
 15. As a restaurant manager, I want to edit menu prices, so that displayed prices remain current.
 16. As a restaurant manager, I want to toggle menu availability, so that unavailable food cannot be ordered.
-

3. Functional Requirements

3.1 Authentication

- FR-01:** The system shall allow a customer to log in using a valid email and password.
 - FR-02:** The system shall authenticate users against the local mock users array.
 - FR-03:** The system shall allow a new user to sign up using name, email, password, confirmation password, and role.
 - FR-04:** The system shall validate the email format before authentication or signup.
 - FR-05:** The system shall require a password of at least eight characters containing at least one digit.
 - FR-06:** The system shall display an error when the password and confirmation password do not match.
 - FR-07:** The system shall display a loading indicator while authentication is being simulated.
-

3.2 Menu

- FR-08:** The system shall display menu items from local mock menu data.
 - FR-09:** The system shall display menu items under Starters, Mains, Desserts, and Drinks categories.
 - FR-10:** The system shall display a Daily Special badge for items marked as special.
 - FR-11:** The system shall disable the Add to Cart button for unavailable items.
 - FR-12:** The system shall visually distinguish unavailable menu items from available items.
 - FR-13:** The system shall allow customers to select a menu category.
-

3.3 Search

- FR-14:** The system shall allow customers to search menu items by name.
 - FR-15:** The system shall delay search execution using debounce functionality.
 - FR-16:** The system shall display a friendly empty-state message when no menu item matches the search.
 - FR-17:** The system shall allow customers to sort menu items by price or name.
-

3.4 Cart

- FR-18:** The system shall allow customers to add available menu items to the cart.
- FR-19:** The system shall allow customers to increase the quantity of a cart item.
- FR-20:** The system shall allow customers to decrease the quantity of a cart item.

- FR-21:** The system shall remove an item when its quantity reaches zero.
- FR-22:** The system shall allow customers to remove an item directly from the cart.
- FR-23:** The system shall allow customers to add special instructions to cart items.
- FR-24:** The system shall allow customers to apply valid promo codes.
- FR-25:** The system shall reject invalid promo codes with an error message.
- FR-26:** The system shall allow customers to remove an applied promo code.
-

3.5 Reservation

- FR-27:** The system shall allow customers to select a reservation date.
- FR-28:** The system shall provide hourly reservation slots from 12:00 to 22:00.
- FR-29:** The system shall disable a time slot when no suitable table is available.
- FR-30:** The system shall validate that the reservation date is not in the past.
- FR-31:** The system shall validate party size between 1 and 12 guests.
- FR-32:** The system shall validate Pakistani mobile numbers using the format 03XX-XXXXXXX .
- FR-33:** The system shall require reservations to be at least one hour ahead of the current time.
- FR-34:** The system shall allow customers to select an available table.
- FR-35:** The system shall display a reservation confirmation before saving the booking.
- FR-36:** The system shall allow customers to cancel an existing reservation.
-

3.6 Orders

- FR-37:** The system shall allow customers to choose Dine-in or Takeaway.
- FR-38:** The system shall require a table selection for Dine-in orders.
- FR-39:** The system shall require a pickup time for Takeaway orders.
- FR-40:** The system shall create an order with an ID, items, total, type, status, and timestamp.
- FR-41:** The system shall create new orders with Pending status.
- FR-42:** The system shall calculate subtotal, service charge, tax, discount, and grand total.
- FR-43:** The system shall display an Order Summary before placing an order.
- FR-44:** The system shall display an Order Tracking screen after a successful order.
- FR-45:** The system shall update the demo order status from Pending to Preparing, Ready, and Served.
-

3.7 Manager Dashboard

- FR-46:** The system shall display the Manager Dashboard only to users with the manager role.

FR-47: The system shall display incoming orders to the manager.

FR-48: The system shall allow the manager to update order status.

FR-49: The system shall display customer reservations to the manager.

FR-50: The system shall allow the manager to confirm or decline reservations.

FR-51: The system shall allow the manager to add menu items.

FR-52: The system shall allow the manager to edit menu item prices.

FR-53: The system shall allow the manager to toggle menu item availability.

FR-54: The system shall update customer menu data when a manager changes menu availability or pricing.

4. Non-Functional Requirements

4.1 Usability

The system shall provide a simple and understandable interface for customers and managers. Buttons, labels, status indicators, forms, and navigation controls shall be clearly visible and readable.

4.2 Performance

Menu lists shall use FlatList for efficient rendering and scrolling. Search shall use debounce to avoid unnecessary filtering during every keystroke. Expensive derived calculations such as order totals shall use useMemo where appropriate.

4.3 Responsiveness

The interface shall adapt to different mobile screen sizes and orientations supported by React Native. Layouts shall use flexible dimensions, spacing, and scrollable containers rather than fixed desktop-only dimensions.

4.4 Maintainability

The source code shall be organized into reusable folders including:

- `src/components`
- `src/screens`
- `src/context`
- `src/reducers`
- `src/hooks`
- `src/data`
- `src/navigation`

Reusable components and custom hooks shall prevent unnecessary duplication.

4.5 Data Handling

Because no backend is used, application data shall be stored in local JavaScript/mock data files and React state. Important changes such as orders, reservations, and menu edits shall use AsyncStorage for local persistence.

4.6 Reliability

The system shall provide loading, validation, error, and empty states where required. Timers and intervals shall be cleared during component cleanup.

4.7 Theme Consistency

All screens shall obtain colors from ThemeContext so that switching between light and dark mode updates the application consistently.

5. Client-Side Data Model (Mock Data)

Data Set Name	Fields	Description
Users	id, name, email, password, role	Local customer and manager login accounts.
Categories	id, name	Menu categories such as Starters, Mains, Desserts, and Drinks.
MenuItems	id, name, description, price, category, image, isSpecial, isAvailable	Restaurant food and drink offerings.
Tables	id, seats, status	Restaurant tables and their seating capacity.
Reservations	id, customer, date, time, guests, tableId, phone, status	Customer table booking information.
Orders	id, items, total, type, status, timestamp	Customer order information and progress.
CartItem	menuItem, quantity, note	Items currently selected by the customer.
Promo Codes	code, discountPercent	Local promotional codes used for cart discounts.

The application does not use a remote database. Mock data is imported from local JavaScript files and managed through React state, Context API, reducers, and custom hooks.

6. UML Diagrams

6.1 Use Case Diagram

Figure 1 — Savoria Restaurant App Use Case Diagram

The Use Case Diagram shows the interaction between the Customer and Restaurant Manager actors and the major functions provided by the Savoria Restaurant App. Customer activities include browsing the menu, searching, managing the cart, making reservations, placing orders, and tracking orders. Manager activities include managing orders, reservations, and menu items.

Diagram file: A1/UML/Use_Case_Diagram.png

6.2 Class Diagram

Figure 2 — Savoria Restaurant App Class Diagram

The Class Diagram models the main entities of the Restaurant App, including User, Customer, Manager, Category, MenuItem, Cart, CartItem, Order, Reservation, and Table. It shows attributes, operations, inheritance, relationships, and multiplicities between these classes.

Diagram file: A1/UML/Class_Diagram.png

6.3 Sequence Diagram

Figure 3 — Customer Places an Order Sequence Diagram

The Sequence Diagram shows the frontend interaction when a customer adds a menu item to the cart and proceeds through checkout until the Order Tracking screen is displayed. It demonstrates interactions among the menu screen, cart context, reducers, order summary, orders context, and order tracking screen.

Diagram file: A1/UML/Sequence_Diagram.png

6.4 State Machine Diagram

Figure 4 — Order Lifecycle State Machine Diagram

The State Machine Diagram represents the lifecycle of an order. The order begins in Pending status and can move to Preparing, Ready, and Served. Cancellation is possible from Pending and Preparing states. Served and Cancelled are terminal states.

Diagram file: A1/UML/State_Machine_Diagram.png

6.5 Component Diagram

Figure 5 — Savoria Restaurant App Component Diagram

The Component Diagram shows the frontend architecture of the React Native application. It represents screens, Context Providers, reducers, custom hooks, reusable components, local mock data, and Expo Router navigation. The diagram demonstrates how these frontend components depend on each other.

Diagram file: A1/UML/Component_Diagram.png

7. MVP Frontend Development (React Native)

7.1 Login and Signup Screen

Purpose

The Login and Signup screen provides local authentication for customers and restaurant managers.

UI Elements

- Email input
- Password input
- Full name input for signup
- Confirm password input
- Customer/Manager role selector
- Password visibility toggle
- Submit button
- ActivityIndicator
- Validation messages
- Login/Signup mode switch

Navigation Entry

The screen is entered when the application requires authentication.

Navigation Exit

A successful customer login navigates to the customer menu/explore screen. A successful manager login navigates to the Manager Dashboard.

Local Data

```
src/data/users.js
```

Hooks

useState: manages form values, errors, password visibility, selected role, and submitting state.

useContext: accesses AuthContext.

useForm: provides reusable form state and validation logic.

7.2 Menu Browsing Screen

Purpose

The Menu screen allows customers to browse restaurant food and drinks.

UI Elements

- Search bar
- Category chips
- Menu item cards
- Daily Special badge
- Price
- Description
- Availability indicator
- Add to Cart button
- Favourite button
- Sort controls
- Loading indicator
- Retry button

- Empty state
- Back-to-top button

Navigation Entry

Customer enters the screen after successful login or through the customer navigation.

Navigation Exit

Customer can navigate to Cart, Profile, Reservations, Order Summary, or Order Tracking.

Local Data

```
src/data/menu.js
```

Hooks

useState: selected category, search state, loading state, favourites, sorting, and UI state.

useEffect: loads mock menu data and handles refresh/lifecycle effects.

useRef: controls the search input, FlatList, debounce timer, previous search, and render counter.

useMemo: calculates filtered and sorted menu data.

useCallback: provides stable menu action handlers.

useDebounce: delays search execution.

7.3 Search and Scroll Controls

Purpose

This functionality allows customers to search menu items efficiently and navigate long menu lists.

UI Elements

- Search input
- Search icon
- Clear button
- Recent search suggestions
- Empty search result message
- FlatList
- Back-to-top floating button
- Debug render counter

Navigation Entry

Search functionality is available directly from the Menu screen.

Navigation Exit

Customers remain on the Menu screen while filtering results.

Local Data

Menu mock data.

Hooks

useRef: stores input references, FlatList reference, timeout values, previous query, and render count.

useState: stores search text, scroll position, and recent search terms.

useDebounce: performs delayed search processing.

7.4 Profile and Theme Screen

Purpose

The Profile screen displays authenticated user information and provides theme controls.

UI Elements

- User avatar
- Name
- Email
- Role
- Dark Mode switch
- Logout button

Navigation Entry

Customer or manager can open Profile from the application navigation.

Navigation Exit

The user can return to the previous screen or logout to the Login screen.

Local Data

Current user from AuthContext.

Hooks

useAuth: provides current user and logout functionality.

useTheme: provides current theme and theme switching.

7.5 Cart Screen

Purpose

The Cart screen allows customers to review and modify selected menu items before checkout.

UI Elements

- Cart item cards
- Quantity stepper
- Remove button
- Special instructions field
- Promo code input
- Discount display

- Item count
- Checkout button

Navigation Entry

Customer opens Cart after adding menu items.

Navigation Exit

Customer proceeds to Order Summary or returns to Menu.

Local Data

CartContext and local menu data.

Hooks

useReducer: manages complex cart state transitions.

useContext: accesses CartContext.

7.6 Order Summary Screen

Purpose

The Order Summary screen calculates and displays the final order cost before placement.

UI Elements

- Cart items
- Subtotal
- Service charge
- Sales tax
- Promo discount
- Grand total
- Dine-in/Takeaway selection
- Table or pickup-time selection
- Place Order button

Navigation Entry

Customer enters the screen from Cart.

Navigation Exit

Successful order placement navigates to Order Tracking.

Local Data

CartContext and OrdersContext.

Hooks

useMemo: calculates subtotal, service charge, tax, discount, and grand total.

useCallback: maintains stable event handlers where required.

useContext: accesses cart and order state.

7.7 Table Reservation Screen

Purpose

The Reservation screen allows customers to reserve available restaurant tables.

UI Elements

- Date selector
- Hourly time slots
- Guest count
- Table selection
- Contact phone
- Validation messages
- Confirmation modal
- My Reservations list
- Cancel button

Navigation Entry

Customer opens the Reservations screen from customer navigation.

Navigation Exit

Customer returns to the previous screen after booking or cancellation.

Local Data

```
src/data/tables.js
```

```
src/data/reservations.js
```

Hooks

useReservation: manages reservation business logic including availability, validation, creation, and cancellation.

useForm: manages reservation form values and validation.

useContext: accesses shared application state where required.

7.8 Order Tracking and Manager Dashboard

Order Tracking Purpose

The Order Tracking screen shows the current order lifecycle and progress.

UI Elements

- Order ID
- Order type
- Order total
- Current status
- Progress indicator

- Elapsed time
- Order timestamp

Order Tracking Navigation

The screen is entered after successful order placement.

Local Data

OrdersContext.

Hooks

useEffect: controls order progression timers and cleanup.

useState: manages elapsed time and screen state.

useContext: accesses OrdersContext.

Manager Dashboard Purpose

The Manager Dashboard provides restaurant managers with tools to manage incoming orders, reservations, and menu items.

UI Elements

Incoming Orders

- Order list
- Customer/order information
- Status controls
- Status update buttons

Reservations

- Reservation list
- Confirm button
- Decline/cancel button

Menu Management

- Add menu item
- Edit price
- Toggle availability

Navigation Entry

The Manager Dashboard is available only when the authenticated user's role is `manager`.

Navigation Exit

Manager can navigate between Incoming Orders, Reservations, and Menu Management.

Local Data

- OrdersContext
- MenuContext
- Reservation mock data

- Menu mock data

Hooks

useContext: accesses authentication, orders, menu, and theme state.

useReducer: manages order state.

useEffect: handles local persistence and loading.

useCallback: provides stable manager action handlers.

8. React Hooks Used in the MVP

Hook	Usage
useState	Form state, filters, theme UI, search, favourites, reservation values, and order type.
useEffect	Data loading, timers, persistence, cleanup, and lifecycle operations.
useRef	Search input, FlatList, timers, previous query, and render counter.
useContext	Authentication, theme, cart, menu, and order state.
useReducer	Cart and order state transitions.
useMemo	Filtered menu data and order total calculations.
useCallback	Stable event handlers for optimized components.
React.memo	Prevents unnecessary MenuItemCard re-renders.
Custom Hooks	Reusable form, debounce, and reservation business logic.

9. Frontend Data and State Management

The application uses React state management rather than a third-party state management library.

The major shared states are:

- Authentication state through AuthContext.
- Theme state through ThemeContext.
- Cart state through CartContext and cartReducer.
- Menu state through MenuContext.
- Orders state through OrdersContext and ordersReducer.

Local mock files provide initial application data.

AsyncStorage provides local persistence for supported application data.

10. Frontend-Only Architecture

Savoria is intentionally implemented as a frontend-only MVP.

The application uses:

- React Native
- Expo
- Expo Router
- React Context API
- React reducers
- React Hooks
- Custom Hooks
- Local JavaScript mock data
- AsyncStorage

There is no remote backend, database, payment server, or external API.

The architecture is designed to demonstrate frontend application structure, state management, reusable logic, navigation, validation, and UI interaction.

11. Conclusion

The Savoria Restaurant App MVP provides the core customer and restaurant manager workflows required by Assignment No. 1.

Customers can authenticate, browse and search the menu, manage a cart, apply promotional codes, reserve tables, place orders, and track order progress.

Restaurant managers can manage incoming orders, reservations, and menu availability.

The application demonstrates the required React Native hooks and frontend architecture while remaining within the frontend-only scope of the assignment.